ПРОГРАММИРОВАНИЕ И ОБРАБОТКА ГРАФИЧЕСКОГО ИНТЕРФЕЙСА

Основы тестирования

Лекция 4: проверяем код кодом, а не кликами

Слайды + живой код (TestingDemo: кликер и NUnit)

Фоменко М.Ю.

МАРШРУТ КУРСА

Где мы находимся

Было

Лекция 3: интерфейсы, DI, события.

ЛР2: BigNumber, игрок, противник.

ЛР4: IEnemy, ISaveList<T>.

Сегодня

Модульные тесты на NUnit.

Какие тесты писать.

Моки, покрытие, мутации.

LLM и тесты.

Применяем

ЛР6: тесты на функции и на BigNumber.

Курсовая: тесты на логику.

Интерфейсы из лекции 3 сегодня окупятся: именно они делают код тестируемым.

КАРТА ЛЕКЦИИ

Шесть блоков и примерное время

1. Методология
~10 мин

Ошибка и сбой, пирамида, три принципа ISTQB

2. Анатомия теста
~25 мин

NUnit, AAA, Assert.That, TestCase, исключения

3. Какие тесты писать
~15 мин

Классы эквивалентности и граничные значения

4. Дублёры
~15 мин

Random и файлы за интерфейсом, stub и spy

5. Хорош ли тест
~10 мин

FIRST, покрытие, мутационное тестирование

6. LLM и тесты
~15 мин

Тест-дизайн, проблема оракула, агент в CI

UI-тестов («нажать кнопку — проверить окно») сегодня нет: это отдельная тема. Тестируем код.

ПРОБЛЕМА

«Я же проверял»

Поправили перенос разряда в BigNumber.Add. Проверили руками 999 + 1 — теперь 1000. А остальное?

?

Вычитание
использует тот же перенос

?

Деление
внутри вычитает

?

Цена апгрейда
на 25-м уровне

?

Сохранение
пишет числа в JSON

Честный ответ — вы не знаете. Узнать можно только повторив все проверки руками. После каждой правки.

ПРОБЛЕМА

Баг, который не видно глазами

```
public virtual void TakeDamage(BigNumber damage)
{
    CurrentLife.Subtract(damage);
}
```

Что не так

BigNumber неизменяемый: Subtract возвращает новый объект. Результат выброшен.

Что видно

Ничего. Компилируется, предупреждений нет. Здоровье противника не меняется.

Тест на этот метод — пять строк. Он падает сразу и показывает, где именно.

1

Немного методологии

Ровно столько, чтобы говорить на одном языке с тестировщиками

ТЕРМИНЫ ISTQB

Ошибка → дефект → сбой

Ошибка

error, mistake

Человек забыл, что BigNumber неизменяемый

→

Дефект

defect, bug

Строка CurrentLife.Subtract(damage); без присваивания

→

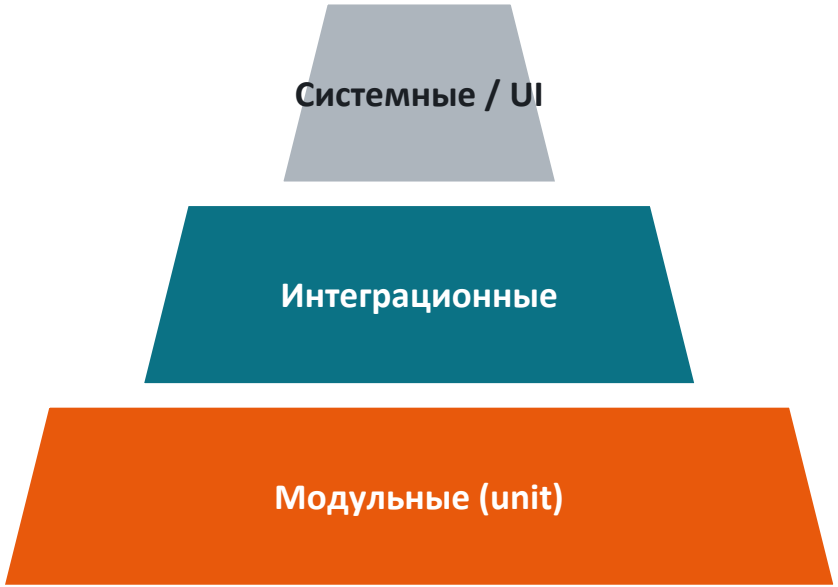
Сбой

failure

Слизень не умирает, сколько ни кликай

Тест ловит сбой. По сбою находим дефект. Отладчик помогает найти, где он сидит.

Пирамида тестов



Запустили игру, прошли бой

Сохранили в JSON — загрузили

BigNumber.Add, TakeDamage

мало
медленные, хрупкие

много
быстрые, точные

Ещё есть приёмочное тестирование: соответствует ли программа ожиданиям заказчика. В курсе — защита лабы по чек-листу.

Три принципа на сегодня

1. **Тесты показывают наличие дефектов, а не их отсутствие.** Зелёные тесты = нет тех багов, которые проверили.

2. **Исчерпывающее тестирование невозможно.** Пар чисел для BigNumber.Add бесконечно много — выбираем с умом (блок 3).

3. **Раннее тестирование экономит время.** Баг через минуту после написания чинится за минуту, на защите — за вечер.

Всё, что формулируется как «при таких данных — такой результат», отдаём машине. Особенно регрессию.

2

Анатомия модульного теста

NUnit, AAA, проверки, параметры, исключения

ИНСТРУМЕНТ

NUnit, MSTest, xUnit — одно и то же

	NUnit	MSTest	xUnit
Класс с тестами	`[TestFixture]`	`[TestClass]`	— не нужен
Тест	`[Test]`	`[TestMethod]`	`[Fact]`
С параметрами	`[TestCase(1, 2)]`	`[DataRow(1, 2)]`	`[InlineData(1, 2)]`
Перед каждым	`[SetUp]`	`[TestInitialize]`	конструктор
Проверка	`Assert.That(x, Is.EqualTo(5))`	`Assert.AreEqual(5, x)`	`Assert.Equal(5, x)`

Идеи одинаковые, отличаются названия атрибутов. Мы берём NUnit — на нём Лаба 6.

СТРУКТУРА РЕШЕНИЯ

Тесты живут в отдельном проекте

```
TestingDemo.sln
├─ Clicker.Core
│     BigNumber, CPlayer, противники
└─ Clicker.Tests
       ссылка на Clicker.Core
       BigNumberTests, EnemyTests…
```

Пакеты NuGet

- NUnit — сам фреймворк
- NUnit3TestAdapter — без него VS не видит тесты
- Microsoft.NET.Test.Sdk — запуск тестов
- Шаблон NUnit Test Project ставит их сам; dotnet new nunit

В обычном консольном проекте (как в Лабе 6) первые два пакета ставятся через NuGet вручную.

ВНИМАНИЕ: NUNIT 4+

Assert.AreEqual больше нет

Старый стиль — ошибка компиляции

```
Assert.AreEqual(4, result);
Assert.IsTrue(defeated);
```

Работает в NUnit 3, 4 и 5

```
Assert.That(result, Is.EqualTo(4));
Assert.That(defeated, Is.True);
```

```
error CS0117: 'Assert' does not contain a definition for 'AreEqual'
```

Быстрый выход: ClassicAssert.AreEqual(…) и using NUnit.Framework.Legacy; Правильный: Assert.That.

Шаблоны и ИИ-ассистенты часто пишут старый стиль: его в интернете больше всего.

ЗОЛОТОЕ ПРАВИЛО

Arrange — Act — Assert

Arrange
подготовить

Act
один вызов

Assert
проверить

```
[Test]
public void TakeDamage_LessThanLife_ReducesLife()
{
    var slime = new CNormalEnemy("Слизень",
        new BigNumber("100"), new BigNumber("10"));

    bool defeated = slime.TakeDamage(new BigNumber("30"));

    Assert.That(slime.CurrentLife.ToString(), Is.EqualTo("70"));
    Assert.That(defeated, Is.False);
}
```

Одно действие на тест. Если Act — пять вызовов, при падении непонятно, какой из них виноват.

КРАСНЫЙ ТЕСТ

Упавший тест объясняет сам

✕ **TakeDamage_LessThanLife_ReducesLife**

```
Expected: "70"
But was:  "100"
```

✓ TakeDamage_Negative_Throws…
✕ TakeDamage_EqualToLife_Enemy…

Что видно без отладчика

- Какой тест упал — по имени
- Что ожидалось и что получилось
- Здоровье не изменилось вообще — значит, результат вычитания потерян

Зелёный тест — хорошо. Красный тест — тоже хорошо: он нашёл баг за вас.

ПРОВЕРКИ

Assert.That: фактическое, потом ограничение

Проверка	Пример
Равно	`Assert.That(level, Is.EqualTo(2));`
Истина / ложь	`Assert.That(defeated, Is.True);`
Не null	`Assert.That(enemy, Is.Not.Null);`
Больше	`Assert.That(level, Is.GreaterThan(1));`
Строка содержит	`Assert.That(ex.Message, Does.Contain("ноль"));`
Коллекция	`Assert.That(enemies, Has.Count.EqualTo(3));`

double: 1.1 × 1.1 × 1.1 = 1.3310000000000004. `Is.EqualTo(1.331).Within(1e-9)`

Порядок: сначала фактическое, потом ожидаемое. Перепутаете — сообщение об ошибке будет врать.

ПАРАМЕТРЫ

Один тест — много сценариев: [TestCase]

```
[TestCase("2", "2", "4")]
[TestCase("999", "1", "1000")]   // перенос
[TestCase("3712117", "15512900", "19225017")]
[TestCase("0", "0", "0")]
public void Add_TwoNumbers_ReturnsSum(
    string a, string b, string expected)
{
    var result = new BigNumber(a) + new BigNumber(b);

    Assert.That(result.ToString(), Is.EqualTo(expected));
}
```

Обозреватель тестов

✓ ("2","2","4")
✕ ("999","1","1000")
✓ ("3712117",…)
✓ ("0","0","0")

Упал только перенос — значит, сломан перенос, а не сложение вообще.

Assert.Throws: правильный ответ — ошибка

```
[Test]
public void Divide_ByZero_ThrowsDivideByZeroException()
{
    var number = new BigNumber("100");
    var zero = new BigNumber("0");

    var ex = Assert.Throws<DivideByZeroException>(() => { _ = number / zero; });

    Assert.That(ex.Message, Does.Contain("ноль"));
}
```

Лямбда () => обязательна

Без неё исключение вылетит раньше, чем Assert.Throws успеет его поймать.

Тест красный, если…

исключения не было вовсе или оно другого типа.

ПОДГОТОВКА

[SetUp] или вспомогательный метод

[SetUp] — общий объект для всех

```
private CNormalEnemy _slime;

[SetUp]
public void CreateEnemy()
{
    _slime = new CNormalEnemy(
        "Слизень", new BigNumber("100"),
        new BigNumber("10"));
}
```

Метод-помощник — данные видны в тесте

```
static CPlayer PlayerWithGold(string g) =>
    new(new BigNumber(g),
        damage: new BigNumber("1"),
        upgradeCost: new BigNumber("100"));

[Test]
public void TryUpgrade_NotEnoughGold_…()
{
    var player = PlayerWithGold("99");
```

Главное — каждый тест получает свежие данные. Общий объект на весь класс = тесты зависят от порядка запуска.

Почему два одинаковых BigNumber «не равны»

```
var sum = new BigNumber("5") + new BigNumber("5");
Assert.That(sum, Is.EqualTo(new BigNumber("10")));    // красный!
```

Почему

Equals не переопределён — сравнение по ссылке: «это один объект в памяти?»

Как в ЛР6

Сравнивать строки: result.ToString() с "10".

Как в индустрии

Переопределить Equals и GetHashCode (IEquatable<BigNumber>) — сравнение по значению.

То же касается игрока, противника и любого вашего класса.

ИМЕНА

Метод_Сценарий_Результат

Плохо	Хорошо
`Test1`	`TryUpgrade_NotEnoughGold_ReturnsFalse`
`TestTakeDamage`	`TakeDamage_MoreThanLife_LifeDoesNotGoNegative`
`DivideTest2`	`Divide_ByZero_ThrowsDivideByZeroException`

Упал TakeDamage_MoreThanLife_LifeDoesNotGoNegative — и уже понятно, что здоровье ушло в минус.

Запуск: Тест → Обозреватель тестов → Запустить все. Из консоли и на сервере: dotnet test.

3

Какие тесты писать

Классы эквивалентности и граничные значения

КЛАССЫ ЭКВИВАЛЕНТНОСТИ

Бронированный противник: броня 10

```
var effective = damage > Armor ? damage - Armor : BigNumber.Zero;
```



< 0
ошибка: исключение
представитель: −5

0 … 10
броня поглощает всё
представитель: 5

> 10
проходит урон − броня
представитель: 50

Один представитель на класс: если работает для 50, почти наверняка работает и для 51.

ГРАНИЧНЫЕ ЗНАЧЕНИЯ

Баги живут на границах: 9, 10, 11

```
[TestCase("0",  "100")]   // минимум
[TestCase("9",  "100")]   // граница − 1
[TestCase("10", "100")]   // ровно броня
[TestCase("11", "99")]    // граница + 1
[TestCase("50", "60")]    // представитель
public void TakeDamage_ArmorAbsorbs…(
    string damage, string expectedLife)
```

Где ещё границы

- BigNumber: 999 + 1, 1000 − 1
- Игрок: золота ровно на апгрейд
- Коллекции: пустая, один элемент, много

Поменяют damage > Armor на damage >= Armor — заметит только тест на 10. Тест на 50 промолчит.

ЧЕК-ЛИСТ

Пять вопросов к любому методу

1. **Обычный случай** — то, ради чего метод написан
2. **Границы** — ровно на границе и шаг в обе стороны
3. **Особые значения** — 0, пусто, null, минус, огромное
4. **Ошибочный ввод** — явное исключение, а не тихий мусор
5. **Состояние** — что изменилось и что НЕ должно было

Не тестируем

- private напрямую — через public
- чужой код: List, JsonSerializer
- свойства без логики
- обработчики кнопок — логику выносим в классы

4

Зависимости и дублёры

Random, файлы и время прячем за интерфейс

ПРОБЛЕМА

Не весь код послушный, как BigNumber

Random

Каждый запуск — новое число. Как проверить, что на 0.96 выпадает дракон?

Файлы

Медленно, оставляет мусор, зависит от прав доступа

Время

DateTime.Now в тесте — каждый раз другое

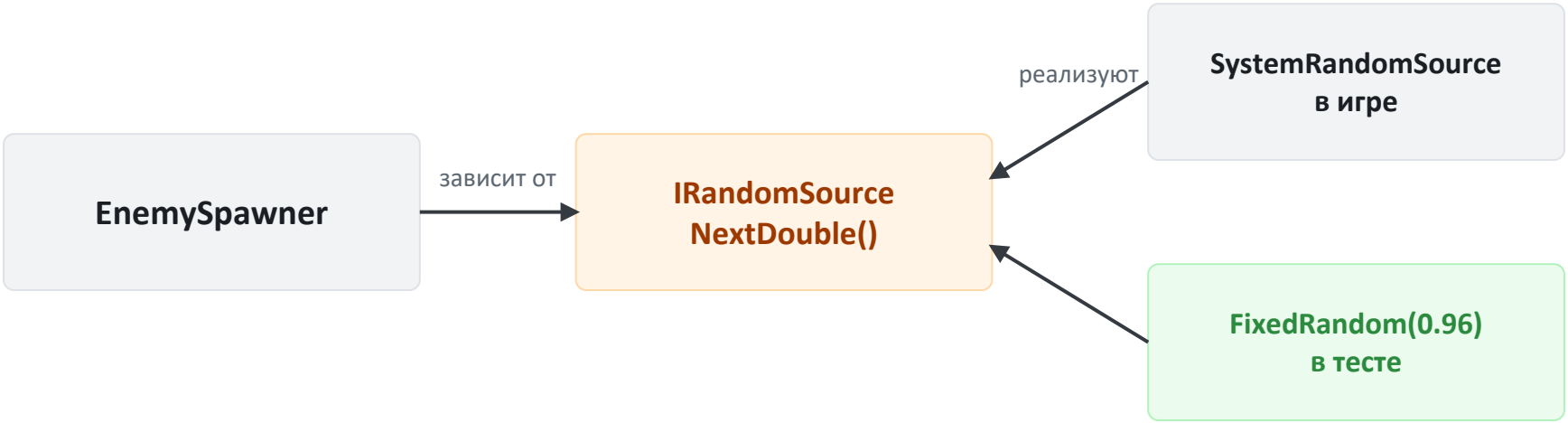
Сеть и БД

Нет интернета — тест красный, хотя код верный

Тест, который то зелёный, то красный, хуже никакого: ему перестают верить.

РЕШЕНИЕ

Зависимость — за интерфейсом




```
public EnemySpawner(IReadOnlyList<EnemyTemplate> templates, IRandomSource random)
// в игре:  new EnemySpawner(templates, new SystemRandomSource())
// в тесте: new EnemySpawner(templates, new FixedRandom(0.96))
```

Пять видов дублёров

Вид	Что делает	Пример
Dummy	Нужен, чтобы скомпилировалось; не используется	null в ненужный параметр
Stub (заглушка)	Возвращает заданные ответы	«Случайность» всегда 0.96
Spy (шпион)	Заглушка, которая запоминает вызовы	Считает вызовы Save
Mock	Сам знает, каких вызовов ждёт, и проверяет их	Создаётся библиотекой
Fake	Рабочая упрощённая реализация	Хранилище в словаре вместо файла

Стаб подсовывает ответ. Шпион и мок проверяют, что вызов был.

STUB

Управляемая случайность

```
class FixedRandom : IRandomSource
{
    double _value;
    public FixedRandom(double v)
        => _value = v;
    public double NextDouble()
        => _value;
}
```

Шансы 70 / 25 / 5

[0 … 0.7] (0.7 … 0.95] (0.95 … 1)

```
[TestCase(0.0,  "Слизень")]
[TestCase(0.7,  "Слизень")]  // граница
[TestCase(0.71, "Рыцарь")]
[TestCase(0.96, "Дракон")]
public void Next_UsesRollToPickTemplate(
    double roll, string expected)
{
    var spawner = new EnemySpawner(
        Templates, new FixedRandom(roll));

    Assert.That(spawner.Next().Name,
                Is.EqualTo(expected));
}
```

Каждый запуск — один и тот же результат. И снова граница: 0.7 — последний шанс слизня.

SPY

Проверяем, что сохранение вызвано

```
class SpySaver : ISaveList<CPlayer>
{
    public int SaveCalls { get; private set; }
    public void Save(CPlayer p, string path) => SaveCalls++;
    public CPlayer Load(string path) => throw new NotSupportedException();
}
```

```
var player = new CPlayer(new("150"), new("1"), new("100"));
var saver = new SpySaver();
var session = new GameSession(player, saver, "save.json");

session.Upgrade();

Assert.That(saver.SaveCalls, Is.EqualTo(1));
```

Второй тест

Золота 0 → апгрейд не прошёл → SaveCalls == 0

Ни одного файла на диске, тест — за миллисекунды.

В индустрии дублёров не пишут руками

Тот же тест с NSubstitute

```
var saver = Substitute.For<ISaveList<CPlayer>>();      // дублёр «из воздуха»
var session = new GameSession(player, saver, "save.json");

session.Upgrade();

saver.Received(1).Save(player, "save.json");           // проверка вызова
```

Не перемокайте

Подменяйте медленное, случайное и внешнее: файлы, время, сеть, Random. Свои простые классы вроде BigNumber подменять не нужно. Пять моков в тесте — он проверяет устройство кода, а не поведение.

5

Хорош ли тест

FIRST, покрытие кода и мутационное тестирование

СВОЙСТВА ХОРОШЕГО ТЕСТА

FIRST

F

Fast

Тысяча тестов — за секунды

I

Independent

Порядок запуска не важен

R

Repeatable

Одинаково везде и всегда

S

Self-validating

Сам говорит «прошёл / упал»

T

Timely

Пишется вместе с кодом

Свойства из «Чистого кода» Роберта Мартина (сформулировали Т. Оттингер и Б. Шухерт).

ТИПИЧНЫЕ ОШИБКИ

Тесты, которые врут

Нет Assert	Всегда зелёный: проверяет только, что код не упал
Логика в тесте	if, циклы, ожидаемое тем же алгоритмом. Тестов на тесты нет
try { } catch { }	Глотает падение: зелёный при любом исходе
Тест «проверить всё»	Упал — и непонятно, что сломано
Зависят от порядка	Один упавший роняет соседей
Рефлексия в private	Ломается от любого рефакторинга

Несколько проверок одного результата — нормально: Assert.Multiple покажет все расхождения разом.

ПОКРЫТИЕ КОДА

100% покрытия ≠ хорошие тесты

100%

покрытия даёт этот тест

```
[Test]
public void Everything()
{
    var p = new CPlayer(…);
    p.AddGold(new BigNumber("500"));
    p.TryUpgrade();           // и ни одного Assert
}
```

Покрытие показывает, где тесты побывали. Но не что они проверили.

```
dotnet test --collect:"XPlat Code Coverage"
```

МУТАЦИОННОЕ ТЕСТИРОВАНИЕ

Испортить код и посмотреть, заметят ли тесты

Оригинал

```
if (Gold < UpgradeCost)
    return false;
```

Мутант

```
if (Gold <= UpgradeCost)
    return false;
```

Тесты: золота 99 и 150

32 из 32 зелёные. Мутант выжил — тесты не видят разницы.

+ тест: золота ровно 100

Мутант откажет в апгрейде — тест красный. Мутант убит.

Выживший мутант = забытое граничное значение. Инструмент для .NET — Stryker.NET.

6

LLM в тестировании

Где экономит время и где незаметно врёт

ОБЗОР

Три сценария

1. Тест-дизайнер

LLM составляет таблицу кейсов по спецификации. Код не пишет.

Риск низкий

2. Автор тестов

LLM пишет тесты по коду метода. Быстро, но есть проблема оракула.

Риск средний

3. Агент в CI

Агент сам пишет тесты на каждый помеченный коммит и открывает PR.

Нужны ограждения

Во всех трёх ответственность за тест — на человеке. Политика курса та же: использовать можно, объяснять — вам.

СЦЕНАРИЙ 1

LLM как тест-дизайнер

Промпт

```
Метод bool CPlayer.TryUpgrade().
Если Gold < UpgradeCost — false, ничего не меняет.
Иначе списывает цену, Level + 1, удваивает
Damage и UpgradeCost, возвращает true.

Составь таблицу тест-кейсов: класс
эквивалентности, вход, ожидаемый результат.
Отметь граничные значения. Код не пиши.
```

Потом — сами

- Каждый кейс — по спецификации?
- Есть «ровно хватает»?
- Нет дублей одного класса?
- Обратный ход: «что не покрыто?»

Спецификацию, а не код: модель должна знать, как надо, а не как написано.

СЦЕНАРИЙ 2

Проблема оракула

Дали модели метод с багом

```
bool TakeDamage(BigNumber damage)
{
    CurrentLife.Subtract(damage);
    return IsDead;
}
```

Получили зелёный тест

```
var enemy = new CNormalEnemy(…"100"…);

bool result = enemy.TakeDamage(new("30"));

Assert.That(result, Is.False);
// здоровье не проверено
```

Тест по коду фиксирует то, что код делает, а не то, что он должен делать.

Если модель проверит и здоровье, она выведет из кода «100» — и тест закрепит баг как правильное поведение.

ЧЕК-ЛИСТ

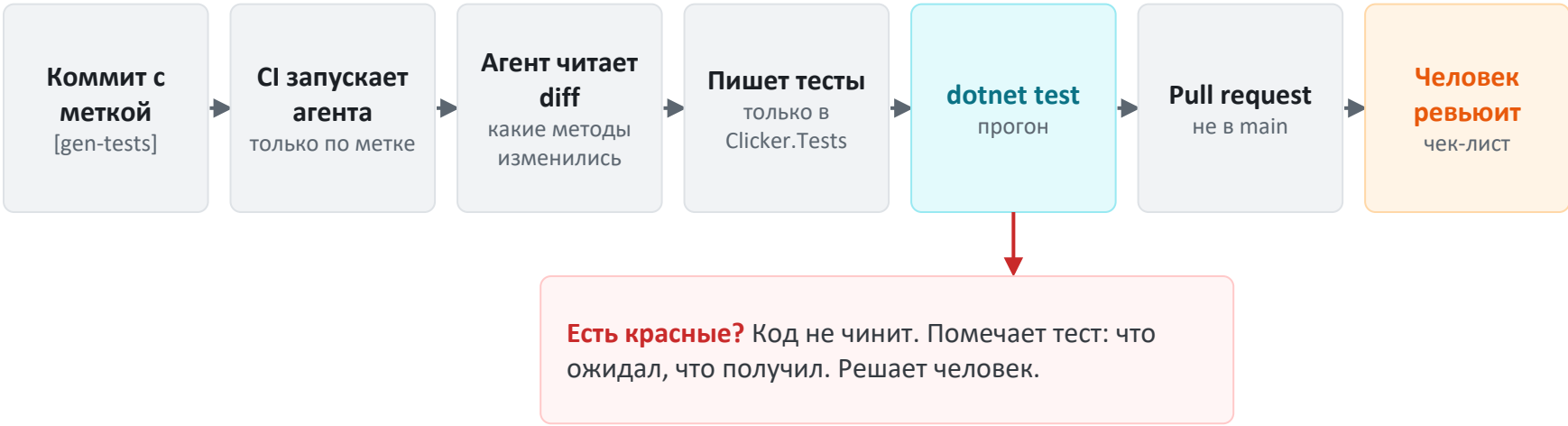
Как принимать сгенерированный тест

1. **Ожидаемое посчитайте сами** по спецификации. Не можете объяснить «19225017» — не принято.
2. **Сломайте код** (< на <=, уберите присваивание) — тест должен покраснеть.
3. **Проверьте API:** Assert.AreEqual в NUnit 4+, выдуманные методы.
4. **Ищите пустые проверки:** нет Assert, мок проверяет сам себя, try/catch.
5. **Смотрите на имена:** Test1 = никто не думал, что проверяется.

Сгенерированный тест — черновик, за который отвечаете вы.

СЦЕНАРИЙ 3

Агент в конвейере сборки (CI)



Метка → diff → тесты → прогон → PR → человек. Инструменты меняются, схема — нет.

ОГРАЖДЕНИЯ

Четыре правила для агента

Только по метке

Агент тратит время и деньги. Не каждый коммит требует тестов.

Только diff и спецификация

Не весь репозиторий: меньше контекста — меньше «улучшений» постороннего.

Меняет только тесты

Красный тест — баг или ошибка теста. Решает человек, не агент.

Результат — PR

Не коммит в main. Ревью по чек-листу сценария 2.

Тесты, фиксирующие текущее поведение без спецификации, — характеризационные (М. Физерс). Страховка, а не доказательство.

ИТОГ ЛЕКЦИИ

Что забираем с собой

1. Тест один раз описывает поведение и дальше проверяет его сам. Главная польза — регрессия.
2. AAA, Assert.That, [TestCase], Assert.Throws. В NUnit 4+ AreEqual — это ClassicAssert.
3. Кейсы — не с потолка: классы эквивалентности + граничные значения.
4. Random, файлы, время — за интерфейс; в тесте — заглушка или шпион.
5. Покрытие — где тесты были. Мутации — что они не заметили.
6. LLM пишет черновики тестов. Ожидаемые значения проверяете вы.

ШПАРГАЛКА

Одной страницей

NUnit

```
[TestFixture]  [Test]  [TestCase(…)]
[SetUp] — перед каждым тестом
Assert.That(x, Is.EqualTo(y))
Is.True / Is.Null / Does.Contain
Is.EqualTo(d).Within(1e-9)
Assert.Throws<T>(() => …)
Assert.Multiple(() => { … })
```

Какие тесты

- Обычный случай
- Граница и ±1 шаг
- 0, пусто, null, минус, огромное
- Ошибка → исключение
- Что не должно было измениться
- Имя: Метод_Сценарий_Результат

Полный конспект с примерами, контрольными вопросами и квизом — в материалах курса: «Лекция 4: Основы тестирования».

К СЛЕДУЮЩЕМУ ЗАНЯТИЮ

Применить

Лаба 6

Функции из задания 1 и BigNumber из ЛР2. Граничные случаи на каждую операцию. Assert.That или ClassicAssert.

Свой кликер

Сломать TakeDamage как на слайде и проверить, что ваш тест это ловит.

Курсовая

Вынести логику из обработчиков кнопок в классы — и покрыть её тестами.

Код демо: лекция 4/TestingDemo — Clicker.Core и Clicker.Tests, 33 теста на NUnit 5.

Вопросы

Основы тестирования